

Test Coverage Guide

Overview

Solo-Git maintains high test coverage standards to ensure code quality and prevent regressions.

Coverage Goals

Overall Targets

- **Overall Coverage:** $\geq 85\%$
- **Core Engines:** $\geq 90\%$
- **CLI Commands:** $\geq 90\%$
- **Critical Paths:** 100%
- **UI Components:** $\geq 85\%$

Current Coverage

Run `pytest --cov=sologit --cov-report=term-missing` to see current coverage.

```
# Quick coverage check
pytest --cov=sologit --cov-report=term-missing

# Detailed HTML report
pytest --cov=sologit --cov-report=html
open htmlcov/index.html
```

Coverage Configuration

Coverage is configured in `.coveragerc` with the following key settings:

- **Source:** `sologit/` package
- **Branch Coverage:** Enabled (measures if/else branches)
- **Omissions:** Tests, examples, GUI code
- **Exclusions:** Debug code, abstract methods, type checking

Running Coverage Tests

Basic Coverage

```
# Run tests with coverage
pytest --cov=sologit

# With missing lines report
pytest --cov=sologit --cov-report=term-missing

# With HTML report
pytest --cov=sologit --cov-report=html
```

Advanced Coverage

```
# Branch coverage
pytest --cov=sologit --cov-branch

# Coverage for specific module
pytest --cov=sologit.engines --cov-report=term-missing

# Fail if coverage below threshold
pytest --cov=sologit --cov-fail-under=85
```

Coverage Reports

```
# Terminal report with missing lines
pytest --cov-report=term-missing

# HTML interactive report
pytest --cov-report=html
open htmlcov/index.html

# XML report (for CI)
pytest --cov-report=xml

# JSON report (for tooling)
pytest --cov-report=json
```

Coverage by Module

Core Engines (Target: $\geq 90\%$)

High priority modules that must have excellent coverage:

- `sologit.engines.git_engine`
- `sologit.engines.patch_engine`
- `sologit.state.manager`
- `sologit.config.manager`

```
# Check core engine coverage
pytest tests/ --cov=sologit.engines --cov-report=term-missing
```

Orchestration (Target: $\geq 85\%$)

AI and workflow orchestration:

- `sologit.orchestration.ai_orchestrator`
- `sologit.orchestration.cost_guard`
- `sologit.orchestration.model_router`
- `sologit.workflows.*`

```
# Check orchestration coverage
pytest tests/ --cov=sologit.orchestration --cov-report=term-missing
```

CLI (Target: ≥ 90%)

Command-line interface:

- `sologit.cli.commands`
- `sologit.cli.main`

```
# Check CLI coverage
pytest tests/ -m cli --cov=sologit.cli --cov-report=term-missing
```

UI (Target: ≥ 85%)

User interfaces:

- `sologit.ui.test_runner`
- `sologit.ui.formatters`

```
# Check UI coverage
pytest tests/ -m "cli or tui" --cov=sologit.ui --cov-report=term-missing
```

Improving Coverage

Finding Untested Code

```
# Generate HTML report
pytest --cov=sologit --cov-report=html

# Open in browser
open htmlcov/index.html

# Navigate to red/yellow lines
# Write tests for those lines
```

Coverage Workflow

1. Identify Gap

```
bash
pytest --cov=sologit --cov-report=term-missing
# Look for modules with low coverage
```

2. Generate Detailed Report

```
bash
pytest --cov=sologit.engines.git_engine --cov-report=html
open htmlcov/index.html
```

3. Write Tests

- Focus on red (untested) lines
- Prioritize branches (if/else paths)
- Target edge cases

4. Verify Improvement

```
bash
```

```
pytest tests/test_new_coverage.py --cov=sologit.engines.git_engine --cov-report=term-
missing
```

Coverage Exclusions

Some code is deliberately excluded from coverage:

```
# Pragma to exclude specific lines
def debug_function(): # pragma: no cover
    print("Debug only")

# Abstract methods are auto-excluded
@abstractmethod
def interface_method(self):
    pass

# Main blocks are auto-excluded
if __name__ == "__main__":
    main()

# Type checking code is auto-excluded
if TYPE_CHECKING:
    from typing import SomeType
```

CI/CD Integration

Coverage in GitHub Actions

Coverage is automatically checked in CI:

```
- name: Run tests with coverage
  run: pytest --cov=sologit --cov-report=xml

- name: Upload to Codecov
  uses: codecov/codecov-action@v4
  with:
    files: ./coverage.xml
```

Coverage Badges

Coverage badges are generated automatically:

[

bahmutov / instrument-example

Unwatch Star 0 Fork

Code Issues 0 Pull requests 0 ZenHub Actions Projects 0 Wiki Settings More

An example application used in my series of videos about Cypress and code coverage <https://www.youtube.com/playlist?list...> Edit

Manage topics

13 commits 1 branch 0 releases 1 contributor 1 week old

Branch: master Create new file Find file Clone or download

bahmutov merge

.github/workflows adding ci

.vscode working

cypress playing

public add public folder

src playing

All checks have passed
2 successful checks

ci / test (push) Details

code-coverage — 75% of statements

Latest commit ddb5873 23 hours ago

Pre-commit Coverage Check

Coverage can be checked before commit:

```
# Manual coverage check
pre-commit run coverage-check --all-files
```

Coverage Anti-patterns

✗ Don't Do

```
# Don't write tests just to hit lines
def test_function_exists():
    assert my_function is not None # Meaningless

# Don't skip important error paths
def risky_function():
    return "happy path" # Error paths not tested

# Don't exclude legitimate code
def important_function(): # pragma: no cover
    # This should be tested!
    pass
```

✓ Do

```
# Test behavior, not existence
def test_function_returns_expected_value():
    result = my_function(5)
    assert result == 25

# Test error paths
def test_function_handles_error():
    with pytest.raises(ValueError):
        risky_function(invalid_input)

# Only exclude non-testable code
def __repr__(self): # pragma: no cover
    return f"Object({self.id})"
```

Coverage Best Practices

1. Write Tests First (TDD)

```
# Write failing test
pytest tests/test_new_feature.py -k test_new_feature

# Implement feature
# ...

# Watch coverage increase
pytest tests/test_new_feature.py --cov=sologit.new_feature
```

2. Incremental Improvement

```
# Baseline
pytest --cov=sologit --cov-report=term-missing | tee coverage-before.txt

# Add tests
# ...

# Compare
pytest --cov=sologit --cov-report=term-missing | tee coverage-after.txt
diff coverage-before.txt coverage-after.txt
```

3. Focus on Critical Paths

Priority for 100% coverage:

- Authentication/authorization
- Data persistence/corruption prevention
- Financial/cost tracking
- Security-sensitive operations

4. Branch Coverage

```
# Enable branch coverage
pytest --cov=sologit --cov-branch --cov-report=term-missing

# Look for "partial" branches
# Add tests for uncovered branches
```

Troubleshooting

Coverage Not Updating

```
# Clean coverage data
rm -f .coverage .coverage.*
rm -rf htmlcov/

# Run fresh
pytest --cov=sologit --cov-report=html
```

Missing Coverage for Module

```
# Verify module is in source
cat .coveragerc | grep source

# Check if omitted
cat .coveragerc | grep omit

# Run with debug
coverage run --debug=trace -m pytest
```

CI Coverage Differs from Local

```
# Use same settings as CI
pytest --cov=sologit --cov-report=xml --cov-report=term-missing

# Check XML output
cat coverage.xml
```

Coverage Tools

Coverage.py

Primary coverage tool:

- Install: `pip install coverage`
- Docs: <https://coverage.readthedocs.io/>

pytest-cov

Pytest plugin (recommended):

- Install: `pip install pytest-cov`
- Docs: <https://pytest-cov.readthedocs.io/>

Codecov

Cloud coverage reporting:

- Web: <https://codecov.io/>
- Integrates with GitHub Actions

Summary

Quick Commands

```
# Basic coverage
pytest --cov=sologit

# Detailed report
pytest --cov=sologit --cov-report=term-missing

# HTML report
pytest --cov=sologit --cov-report=html && open htmlcov/index.html

# Fail if below threshold
pytest --cov=sologit --cov-fail-under=85

# Coverage for specific module
pytest --cov=sologit.engines.git_engine
```

Coverage Targets

Module	Target	Priority
Core Engines	≥ 90%	High
CLI	≥ 90%	High
Orchestration	≥ 85%	Medium
UI	≥ 85%	Medium
Overall	≥ 85%	High

For more information, see:

- `.coveragerc` - Coverage configuration
- `pytest.ini` - Pytest configuration
- `docs/TESTING_GUIDE.md` - Comprehensive testing guide